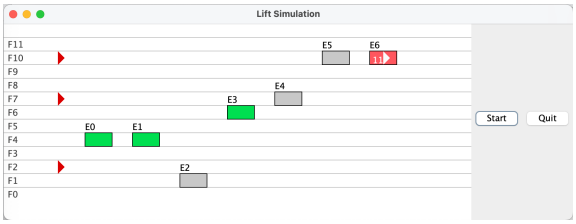
CSC2002S 2025 Concurrency Assignment Lift simulation



1. Objective

In this assignment, your objective is to correct the existing code for a multithreaded Java simulation of lifts in a multi-story building called by passengers and delivering them to another floor.

You will be given a skeleton implementation with a GUI. The simulation runs but violates several rules; **your task is to identify and fix the concurrency issues**, ensuring that it follows defined behavioral rules (no data races, deadlocks, or lost signals). You must use **synchronization mechanisms** to ensure that the simulation adheres to the specified synchronization constraints and maintains safety and liveness. Compound actions on shared state must be correctly protected.

2. Lift simulation

The simulation models a building with F floors and L lifts. Each floor has a single lift button. A press represents a **passenger** waiting at a floor. The GUI shows floors, call indicators for each floor, elevator cars and the current state of each lift: idle (grey), in transit (green), or delivering a passenger (red). Each lift runs in a separate thread. Passengers (run in a single thread) press buttons over time.

The lift simulation starts with a specified F floors, L lifts and C lift calls made by passengers (on the command-line, with defaults if not provided). Lift calls are generated from random floors, and a single idle lift will respond to each call by travelling to the floor (lift is in transit), collect a passenger (lift is in use) and then go to a destination floor (generated randomly too). After this the lift waits (lift is idle) until another call is generated.

You do not need to write the entire simulation from scratch—a skeleton is provided that compiles and runs but does not follow all the rules. Your job is to fix the concurrency in the core class (LiftMonitor) that coordinates lift calls.

3. The behaviour rules for the lift simulation

These rules are listed in a recommended order of trying to enforce them. Tackle these in order; get each rule correct before moving to the next.

1. The **Start button** initiates the simulation. The simulation only begins when **Start** is pressed (no passengers generated, no lift movement before this). *Note that the Quit button terminates the simulation – it does not do this cleanly, but you are not required to fix this.*
2. Pressing a lift button wakes an idle lift. If no lift is idle, the call must persist and **wake the next lift that becomes idle** (no busy waiting). If multiple lifts are idle, only one must answer the lift call.
3. **Safety: no data races.** All shared data must be managed with correct synchronization so the GUI and lifts see consistent state.
4. **No starvation.** Every lift call must be served **eventually**, even under contention.
5. **No busy-waiting** or spinning is allowed.

4. Code Description

The assignment provides a skeleton Java implementation with the following classes:

- LiftSimulation.java: Main class for setting up and starting the simulation, including the GUI.
- Lift.java: Thread class representing the lift.
- Passengers.java: a thread class to generate random lift calls from passengers.
- View.java: JPanel class for visualization of the simulation.
- LiftMonitor.java: The class that handles calling lifts. **This is the only class you will fix and submit,** no other files may be edited.

You will also be given a Makefile to run the program.

You will be provided with versions of all these classes beforehand, except the LiftMonitor.java class. You will also be given a video of how the simulation should run.

For the closed practical, you will correct and submit only the LiftMonitor.java file.

Stay updated with any additional assignment information on the PCP MSTeam. You can ask questions on the class MSteam too.